

Regex Validation Test Report

Secure File Upload System
Project Link : <https://tushar-toc-regex.netlify.app/>

Student	Tushar Kaldate	PRN	202401040191
Roll No.	285	Course	2304220T — Theory of Computation
Topic No.	25	Topic Title	Secure File Upload System
Academic Year	2025–26	Batch/Div	D

This report documents all unit test cases executed against the regex-based validation system. Each of the six form fields is tested with a minimum of three valid and three invalid inputs. The result column indicates PASS (input correctly accepted) or FAIL (input correctly rejected). All test cases are designed to cover boundary conditions, edge cases, and security attack vectors.

1

Test Summary Overview

Field	Regex Pattern (abbreviated)	Valid Tests	Invalid Tests	Total
Username	^(?![0-9]+\$)[a-zA-Z0-9_]{3,20}\$	3	3	6
Email	^[a-zA-Z...]+@[a-zA-Z...]+\.[a-zA-Z]{2,}\$	3	3	6
File Name	^(?!.*\.\w+\.\w+)[a-zA-Z0-9_\-]{1,50}\.(...)\$	3	3	6
File Extension	^\.(jpg jpeg png pdf docx txt)\$	3	3	6
File Size	^\d+(\.\d{1,2})? \$ + range 0<n<=100	3	3	6
Description	^(?!s*\$)[a-zA-Z0-9\s.,!?\-]{10,300}\$	3	3	6
TOTAL		18	18	36

2

Detailed Test Cases — Per Field

Field 1: Username
Regex: `^(?![0-9]+$)[a-zA-Z0-9_]{3,20}$`
Description: Alphanumeric + underscore, 3–20 chars, not all-digits

#	Input String	Expected	Actual	Reason / Observation
1	tushar_285	ACCEPT	PASS ✓	Letters + underscore + digits; 10 chars in range
2	Admin2024	ACCEPT	PASS ✓	Mixed-case letters + digits; 9 chars in range
3	XYZ	ACCEPT	PASS ✓	Minimum 3 uppercase letters; all in character class
4	ab	REJECT	PASS ✓	Only 2 chars — below minimum length of 3
5	123456	REJECT	PASS ✓	All digits — rejected by negative lookahead (?![0-9]+\$)
6	user name!	REJECT	PASS ✓	Space and "!" not in class [a-zA-Z0-9_]

Field 2: Email Address

Regex: `^[a-zA-Z0-9._%+\-]+\@[a-zA-Z0-9.\-]+\.[a-zA-Z]{2,}$`

Description: Standard RFC-like email — local@domain.tld

#	Input String	Expected	Actual	Reason / Observation
1	tushar@college.edu	ACCEPT	PASS ✓	Standard format: local @ domain . tld
2	user.name+tag@sub.dom.in	ACCEPT	PASS ✓	Plus/dot in local; multi-level domain; .in TLD
3	test_2024@gmail.com	ACCEPT	PASS ✓	Underscore in local; well-known TLD
4	@nodomain.com	REJECT	PASS ✓	Empty local part — requires [a-zA-Z0-9._%+\-]+
5	user@.nodot	REJECT	PASS ✓	Domain starts with dot — fails [a-zA-Z0-9.\-]+ rule
6	noatsign.com	REJECT	PASS ✓	Missing "@" separator — DFA halts at q1 without @

Field 3: File Name

Regex: `^(?!.*\.\w+\.\w+)[a-zA-Z0-9_-]{1,50}\.(jpg|jpeg|png|pdf|docx|txt)$`

Description: Safe base name (1–50 chars) + whitelisted extension; blocks double-extension

#	Input String	Expected	Actual	Reason / Observation
1	report_2024.pdf	ACCEPT	PASS ✓	Safe base name; whitelisted .pdf extension
2	photo-001.jpg	ACCEPT	PASS ✓	Hyphen allowed; .jpg in whitelist
3	notes.txt	ACCEPT	PASS ✓	Simple base name; .txt in whitelist
4	malware.pdf.exe	REJECT	PASS ✓	Double-extension attack — blocked by <code>(?!.*\.\w+\.\w+)</code>
5	file name.pdf	REJECT	PASS ✓	Space not in <code>[a-zA-Z0-9_-]</code>
6	script.exe	REJECT	PASS ✓	.exe not in whitelist (jpg jpeg png pdf docx txt)

Field 4: File Extension

Regex: `^\.(jpg|jpeg|png|pdf|docx|txt)$`

Description: Whitelist approach; negative lookahead blocks dangerous extensions

#	Input String	Expected	Actual	Reason / Observation
1	.pdf	ACCEPT	PASS ✓	In whitelist; not in blocklist
2	.png	ACCEPT	PASS ✓	In whitelist; not in blocklist
3	.docx	ACCEPT	PASS ✓	In whitelist; not in blocklist
4	.exe	REJECT	PASS ✓	Executable — blocked by negative lookahead & not in whitelist
5	.sh	REJECT	PASS ✓	Shell script — blocked by blocklist regex
6	pdf	REJECT	PASS ✓	Missing leading dot — fails literal <code>\.</code> match

Field 5: File Size (MB)

Regex: `^d+(\.d{1,2})?$` + range: `0 < value <= 100`

Description: Positive decimal ≤ 100 MB, up to 2 decimal places

#	Input String	Expected	Actual	Reason / Observation
1	4.75	ACCEPT	PASS ✓	Positive decimal; 2 decimal places; within 0–100 range
2	100	ACCEPT	PASS ✓	Integer at max boundary; regex <code>\d+</code> matches
3	0.5	ACCEPT	PASS ✓	Positive; 1 decimal place; within range
4	0	REJECT	PASS ✓	Zero — fails range constraint (must be > 0)
5	100.5	REJECT	PASS ✓	Exceeds max 100 MB — fails range check after regex
6	4.757	REJECT	PASS ✓	3 decimal places — fails <code>(\d{1,2})</code> quantifier

Field 6: Description

Regex: `^(?!s*$)[a-zA-Z0-9\s.,!?\-]{10,300}$`

Description: Safe text 10–300 chars; blocks whitespace-only and injection characters

#	Input String	Expected	Actual	Reason / Observation
1	Project report for TOC 2024.	ACCEPT	PASS ✓	All permitted chars; 30 chars; not whitespace-only
2	Image file from annual survey.	ACCEPT	PASS ✓	Letters, spaces, standard punctuation
3	Financial Q3 - confidential.	ACCEPT	PASS ✓	Hyphen allowed in character class
4	Hi	REJECT	PASS ✓	Only 2 chars — below 10 character minimum
5		REJECT	PASS ✓	Whitespace-only — blocked by <code>(?!s*\$)</code> lookahead

6	alert(1)	REJECT	PASS ✓	<> / chars not in allowed class — XSS blocked
---	----------	--------	--------	---

3 Security & Edge-Case Test Summary

Beyond standard validation, the system is tested against known web-security attack vectors to confirm that regex patterns function as a first line of defence:

Attack / Edge Case	Field Targeted	Test Input	Result	Mechanism
Double-extension attack	File Name	malware.pdf.exe	BLOCKED	Negative lookahead (?!\.*\w+\.\w+)
Executable upload	File Extension	.exe / .bat / .sh	BLOCKED	Whitelist + blocklist regex
XSS injection	Description	<script>alert(1)</script>	BLOCKED	<> not in character class
Whitespace-only input	Description	" " (5 spaces)	BLOCKED	Negative lookahead (?!\s*\$)
All-digit username	Username	987654	BLOCKED	Negative lookahead (![0-9]+\$)
Zero-byte file size	File Size	0	BLOCKED	JS range check: num > 0
Oversized file	File Size	101	BLOCKED	JS range check: num <= 100
Path traversal	File Name	../etc/passwd	BLOCKED	. and / not in [a-zA-Z0-9_-]

Test Outcome: All 36 standard test cases passed (18 correctly accepted, 18 correctly rejected). All 8 security/edge-case tests passed with inputs correctly blocked. The system demonstrates **100% test pass rate** across all validation rules.